Formal Modeling and Analysis of Metric-Timed and Multi-Agent Normative Specifications of Actions

Karam Younes Kharraz

Oral examination for the dissertation submitted to obtain
the Dr. rer. nat degree

29 April 2026

First jury:	Prof. Dr. Martin Leucler
Second jury:	Prof. Dr. Gordon J. Pace
Head of the commission:	Prof. Dr. Stefan Fischer

Motivation: The Cost of Manual Compliance

Digitalization of Normative Systems

Contracts, laws, and collaborative workflows are moving into digital substrates, regulating interactions between humans and autonomous systems.

The Scaling Problem

Regulatory burden costs the global economy trillions of dollars annually: 3.5 % of Europe's GDP. Extracting and enforcing rules manually is error-prone, ad-hoc, and impossible to scale for high-volume automated systems.

- Normative systems are heavily fragmented.
- Overlapping requirements create complex dependency trees.
- How can organizations confidently deploy autonomous agents governed by contradictory or ambiguous rules?

Motivation: The Cost of Manual Compliance

Digitalization of Normative Systems

Contracts, laws, and collaborative workflows are moving into digital substrates, regulating interactions between humans and autonomous systems.

The Scaling Problem

Regulatory burden costs the global economy trillions of dollars annually: 3.5 % of Europe's GDP. Extracting and enforcing rules manually is error-prone, ad-hoc, and impossible to scale for high-volume automated systems.

- Normative systems are heavily fragmented.
- Overlapping requirements create complex dependency trees.
- How can organizations confidently deploy autonomous agents governed by contradictory or ambiguous rules?

Motivation: The Cost of Manual Compliance

Digitalization of Normative Systems

Contracts, laws, and collaborative workflows are moving into digital substrates, regulating interactions between humans and autonomous systems.

The Scaling Problem

Regulatory burden costs the global economy trillions of dollars annually: 3.5 % of Europe's GDP. Extracting and enforcing rules manually is error-prone, ad-hoc, and impossible to scale for high-volume automated systems.

- Normative systems are heavily fragmented.
- Overlapping requirements create complex dependency trees.
- How can organizations confidently deploy autonomous agents governed by contradictory or ambiguous rules?

Research Gap: Why We Need Formal Methods

The Limits of Probabilistic AI

Large Language Models (LLMs) offer high accessibility but lack structural guarantees.

69.7% of their legal outputs still require targeted edits or extensive rework.

The Formal Methods Alternative

Treating specifications as mathematical objects where questions about behavior yield mathematically provable verdicts instead of probabilistic guesses.

Research Gap: Why We Need Formal Methods

The Limits of Probabilistic AI

Large Language Models (LLMs) offer high accessibility but lack structural guarantees.

69.7% of their legal outputs still require targeted edits or extensive rework.

The Formal Methods Alternative

Treating specifications as mathematical objects where questions about behavior yield mathematically provable verdicts instead of probabilistic guesses.

Research Gap: Why We Need Formal Methods

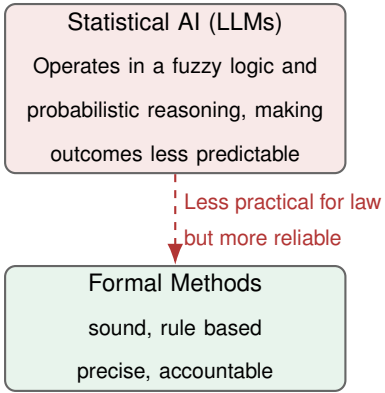
The Limits of Probabilistic AI

Large Language Models (LLMs) offer high accessibility but lack structural guarantees.

69.7% of their legal outputs still require targeted edits or extensive rework.

The Formal Methods Alternative

Treating specifications as mathematical objects where questions about behavior yield mathematically provable verdicts instead of probabilistic guesses.



Thesis Vision: The two families of formal methods applied to Normative reasoning

Thesis Contribution: two formal pipelines for timed normative conflicts and blame-tracking runtime monitors in multi-agent settings.

Before Adoption — Static Analysis

Normative specifications combine obligations, prohibitions, and time constraints.

Problem: Conflicting norms may be *hidden* inside complex temporal conditions, we suggest a novel characterization of timed normative conflicts through quantifiable conflict density measure, quantifies their density, or resolves them automatically.

During Enforcement — Runtime Monitoring

Collaborative contracts require both parties to act in order to *collaborate and not interfere*.

Problem: Standard monitors only report the *first* failure. Open-ended contracts need *cumulative accountability* and per-agent blame attribution over infinite horizons.

Part I

Reasoning about Metric-Timed Normative Conflicts

Micro Metric-Timed Normative Logic μMTNL

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Intuitive understanding of normative conflicts

The Fundamental Intuition

"Normative conflicts occur when complying with one norm can lead to noncompliance with another."

— H. Kelsen, *General Theory of Norms* (1991)

Deontic Conflicts in Dynamic Settings over time

"A deontic conflict occurs when a set of obligations is jointly unsatisfiable over time, meaning that no trace can be constructed that complies with every obligation in the set."

— S. Colombo Tosatto et al., *DEON* (2014)

Conflicts via Satisfiability

"A normative conflict occurs when triggering a normative rule, either always or in some feasible situation, inevitably leads to the violation of one or more rules, because the rule set imposes incompatible requirements."

— N. Feng et al., *ICSE* (2024)

Research Questions — Conflict Analysis

RQ1. **Unsatisfiability and Conflict:** Is every unsatisfiable formula necessarily a conflict? Conversely, does a conflict imply unsatisfiability?

RQ2. **Conflict Quantification:** Can we define a *measure for the degree of conflict* within a specification?

RQ3. **Semantics-Preserving Resolution:** Can we always resolve conflicts without altering the intended compliant behaviours. What are the properties of such resolutions?

Motivation: Inpterpreting Timed norms as a Planning Problem

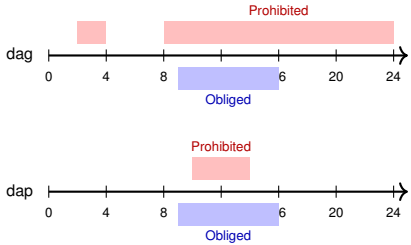
Use case: Goods delivery to a harbour.

- DC: Deliver between 09:00–16:00 to gate **or** parking.
- SR: Parking **prohibited** 10:00–14:00.
- MA: Gate **prohibited** 02:00–04:00, 08:00–24:00.

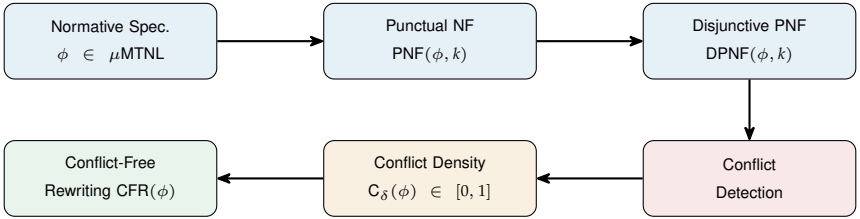
Conflicts are not logical contradictions in isolation: they arise from *overlapping activation windows* and *single-action capacity*.

Conflict at a specific time point t:

1. **Deontic:** Two contradictory obligations/prohibitions specified at the same t.
2. **Ontic:** Two obliged actions cannot be performed at the same time t.



Methodology: The μMTNL Pipeline



- Action-based trace semantics: each timed event carries *exactly one action*.
- Time constraints encoded as **canonical interval sets** $\mathfrak{I}$.
- Normalization decomposes intervals into *punctual atoms*.
- Conflict detection via pattern matching on product terms of the DPNF.
- Structural invariants literal ancestor preservation to ensure no spurious conflicts are introduced.
- Exponential worst-case blowup $\Rightarrow$ mitigated by conflict calculus.

Answering the Research Questions

Q2: Conflict Density

$$\mathsf{C}_\delta(\phi) := \frac{|\text{CPT in DPNF}(\phi)|}{|\text{PT in DPNF}(\phi)|}$$

- $\mathsf{C}_\delta = 0$: conflict-free.
- $\mathsf{C}_\delta = 1$: fully conflicting.
- Use case: $\mathsf{C}_\delta(\text{UC}) \approx 0.81$.

RQ1: Exact Characterisation

ϕ is unsat $\iff$ $\mathsf{C}_\delta(\phi) = 1$

Key results:

- Punctual conflicts = MUS over punctual literals.
- Only deontic & ontic patterns exist.

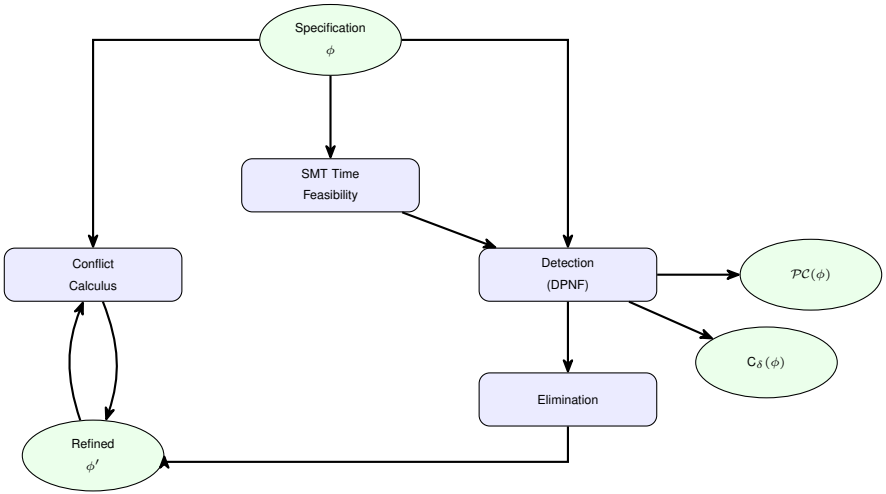
RQ3: Conflict-Free Rewriting

If $\mathsf{C}_\delta(\phi) < 1$:

$\exists\, \phi' \equiv \phi, \quad \mathsf{C}_\delta(\phi') = 0$

- Prune conflicting terms.
- Compactness cost: up to $\mathcal{O}(2^{|\phi|})$.
- Conflict calculus for manual big-step reasoning.

The Conflict Analysis & Resolution Framework



Part II

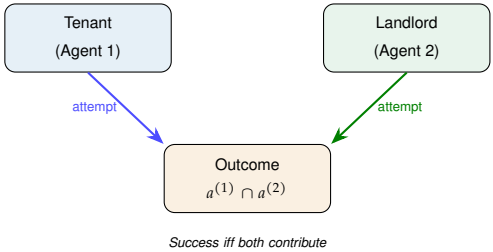
Reasoning about Blame in Multi-Party Collaborative Contracts

Two-Agent Collaborative Normative Logic TACNL

Motivation: Collaborative Contracts & the Accountability Gap

Example: Flat rental agreement.

- *Payment* needs tenant's offer **and** landlord's acceptance.
- *Occupancy* requires landlord to grant access.
- Violations may be *repaired* (pay a fine).
- Contract repeats *every month*, unless terminated with a 3 months notice.

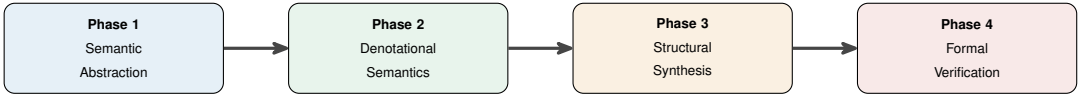


Key challenges:

1. Distinguish **attempts** from **outcomes**.
2. Attribute **blame** (who blocked whom?).
3. Monitor **open-ended** contracts.
4. Provide **cumulative** accountability, not just first-failure.

Formalisation: $\Sigma = \Sigma_C^{(1)} \cup \Sigma_C^{(2)}$
letters $A \in 2^{\Sigma}$, traces $\pi \in (2^{\Sigma})^*$.

Methodology: A Four-Phase Pipeline



- **P1:** Multi-layered trace model with attempt-based interaction ($\Sigma_C^{(i)}$).
- **P2:** Semantics-first for blame and satisfaction: forward looking and quantitative. (4 in total)
- **P3:** Compile TACNL contracts into deterministic Moore/Mealy machines by structural construction.
- **P4:** Conformance proofs: monitor output $\equiv$ denotational semantics for *every* finite prefix.

The Logic TACNL — Syntax

Literals (party $p \in \{1,2\}$, action a):

$$\ell ::= \mathbf{O}_p(a) \mid \mathbf{F}_p(a) \mid \mathbf{P}_p(a) \mid \top \mid \bot$$

Contract operators:

$$C ::= \ell \mid C \wedge C' \mid C\,;\,C' \mid C \blacktriangleright C'$$

Repetition:

C^n (finite) $\quad$ $\mathbf{Rep}(C)$ (unbounded)

Regular expression control:

$\langle\!\langle re \rangle\!\rangle C$ (trigger) $\quad$ $\lceil re \rceil C$ (guard)

Running example:

$C_2 := \mathsf{P}_{\mathsf{Ten}}(\mathsf{OCC})$ (right to occupy)

$C_3 := \mathsf{O}_{\mathsf{Ten}}(\mathsf{PAY\_R}) \blacktriangleright \mathsf{O}_{\mathsf{Ten}}(\mathsf{PAY\_F})$ (pay or fine)

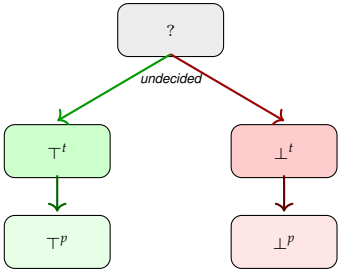
$C := \lceil \Gamma^+ \cdot \mathbf{Notif\_T}^{(1)} \rceil \mathbf{Rep}(C_2 \wedge C_3)$

Five-Valued Forward-Looking Satisfaction Semantics — $\mathbb{V}_5$

Core idea: Identify the *unique decisive prefix* where a contract becomes irrevocably satisfied or violated.

$$\mathbb{V}_5 = \{ \underbrace{?}_{\text{pending}}, \underbrace{\top^t}_{\text{tight sat}}, \underbrace{\bot^t}_{\text{tight viol}}, \underbrace{\top^p}_{\text{post-sat}}, \underbrace{\bot^p}_{\text{post-viol}} \}$$

- $\pi \models_{\top^t} C$: *first* prefix where C is satisfied (acceptance frontier).
- $\pi \models_{\bot^t} C$: *first* prefix where C is violated (rejection frontier).
- $\pi \models_{?} C$: no frontier reached yet.
- $\pi \models_{\top^p} C$ / $\pi \models_{\bot^p} C$: past the decisive point.



Theorem: The five regions are *pairwise disjoint* and *jointly exhaustive*.

Tight Semantics for Contract Operators

Literals (decided in one step):

$$\langle A\rangle \models_{\top^t} \mathbf{O}_1(a) \stackrel{\text{def}}{=} \{a^{(1)}, a^{(2)}\} \subseteq A$$

$$\langle A\rangle \models_{\bot^t} \mathbf{O}_1(a) \stackrel{\text{def}}{=} \{a^{(1)}, a^{(2)}\} \not\subseteq A$$

Conjunction $C \wedge C'$:

- Satisfaction: both succeed; decisive index = $\max(k, k')$.

Sequence $C\,;\,C'$:

- Split witness k: C on $[1,k]$, then C' on $[k{+}1, s]$.

Reparation $C \blacktriangleright C'$:

$$\pi \models_{\top^t} C \blacktriangleright C' \stackrel{\text{def}}{=} \pi \models_{\top^t} C \text{ or } \exists k : \pi_k \models_{\bot^t} C \text{ and } \pi^k \models_{\top^t} C'$$

Repetition $\mathbf{Rep}(C)$:

- $\mathbf{Rep}(C)$ is *never* tightly satisfied on finite traces.

Guard/Trigger:

- $\lceil re \rceil C$: C enforced while re is possible.
- $\langle\!\langle re \rangle\!\rangle C$: C activated once re matches.

Monitor synthesis: Deterministic Moore machine via structural induction.
Conformance: $\lambda_5(q_k) = \mathbb{V}_5$-verdict of $\pi[0,k]$ for all prefixes.

Eleven-Valued Forward-Looking Blame Semantics — $\mathbb{V}_{11}$

Refinement: Split each violation verdict by *who* is responsible.

$$\mathbb{V}_{11} = \underbrace{\{?,\ \top^t,\ \top^p\}}_{\text{unchanged}} \cup \underbrace{\{\bot_S^t,\ \bot_S^p \mid S \subseteq \{1,2\}\}}_{\text{blame-indexed}}$$

Blame rules for literals:

$$\langle A \rangle \vDash_{\bot_p^t} \mathbf{O}_p(a) \stackrel{\text{def}}{=} a^{(p)} \notin A$$

(subject did not attempt)

$$\langle A \rangle \vDash_{\bot_{\bar{p}}^t} \mathbf{O}_p(a) \stackrel{\text{def}}{=} a^{(p)} \in A \wedge a^{(\bar{p})} \notin A$$

(counterparty withheld)

Conjunction blame propagation:

- Both C_1, C_2 violate $\Rightarrow S = S_1 \cup S_2$.

Key properties:

1. Joint blame $\{1,2\}$ arises only via conjunction.
2. Blame monitor $\mathcal{M}_{11}$: same states & transitions as $\mathcal{M}_5$, only $\lambda_5 \to \lambda_{11}$.

Theorem: λ_{11} is a *consistency-preserving refinement* of λ_5.

Limitation: The First-Failure Ceiling

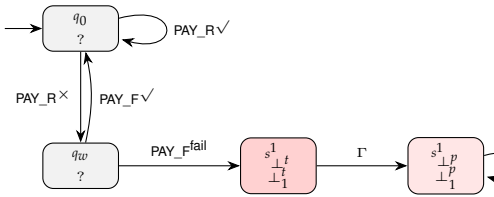
The forward-looking blame monitor detects the **first decisive violation** and then enters a *permanent sink* $\bot_S^p$.

Consequence for $\mathbf{Rep}(C_3)$:

- If tenant is blamed for missing a fine $\Rightarrow$ monitor stays in $\bot_1^p$ forever.
- Even if landlord subsequently blocks a payment, the verdict is *frozen*.

Problem: Open-ended contracts need *cumulative* blame over the full interaction lifespan.

$\Longrightarrow$ **Solution:** Quantitative semantics with scalar and vector domains.



Blame monitor for $\mathbf{Rep}(C_3)$: once $\bot_1^p$ is reached, it *never* leaves.

Contract Progress Monitor — Prog

Key idea: A *derivative operator* that computes the **reachable residual contract** after each event.

$$\text{Prog} : \Gamma^* \times \text{TACNL} \to \text{TACNL} \cup \{\epsilon_C\}$$

Selected clauses:

$$\text{Prog}(\langle A \rangle,\ \ell) := \epsilon_C \quad \text{(literal consumed)}$$

$$\text{Prog}(\langle A \rangle,\ C_1 \wedge C_2) := \text{Prog}(\langle A \rangle, C_1) \wedge \text{Prog}(\langle A \rangle, C_2)$$

$$\text{Prog}(\langle A \rangle,\ \mathbf{Rep}(C)) := \text{Prog}(\langle A \rangle, C)\ ;\ \mathbf{Rep}(C)$$

Reparation (semantic branching):

$$\text{Prog}(\langle A \rangle,\ C_1 \blacktriangleright C_2) := \begin{cases} \text{Prog}(\langle A \rangle, C_1) \blacktriangleright C_2 & \text{if } \langle A \rangle \models_? C_1 \\ \text{Prog}(\langle A \rangle, C_2) & \text{if } \langle A \rangle \models_{\perp^t} C_1 \\ \epsilon_C & \text{if } \langle A \rangle \models_{\top^t} C_1 \end{cases}$$

Trace example on $\mathbf{Rep}(C_3)$:

Month	1	2
Event	$\{\text{PAY\_R}^{(12)}\}$	$\{\text{OCC}^{(1)}\}$
Residual	$\mathbf{Rep}(C_3)$	$\mathbf{O}_1(\text{PAY\_F})\ ;\ \mathbf{Rep}(C_3)$

Finiteness: RRC(C) is finite ⇒ Mealy machine is *finite-state*.

Quantitative Violation Semantics — Singleton Domain $\mathbb{N}$

Transition: From "Did it fail?" to "How many times did it fail?"

$$[\![\cdot]\!]_{\text{qviol}} : \Gamma^* \times \text{TACNL} \to \mathbb{N}$$

Recursive accumulation:

$$[\![\langle A\rangle \circ \pi,\ C]\!]_{\text{qviol}} := \underbrace{[\![\langle A\rangle,\ C]\!]_{\text{qviol}}}_{\text{instant penalty}} + \underbrace{[\![\pi,\ \text{Prog}(\langle A\rangle, C)]\!]_{\text{qviol}}}_{\text{suffix score}}$$

Instantaneous score:

$$[\![\langle A\rangle,\ C]\!]_{\text{qviol}} := \begin{cases} [\![\langle A\rangle, C_1]\!]_{\text{qviol}} + [\![\langle A\rangle, C_2]\!]_{\text{qviol}} & C = C_1 \wedge C_2 \\ 1 & \langle A\rangle \models_{\bot^t} \text{LH}(C) \\ 0 & \text{otherwise} \end{cases}$$

Key properties:

- $[\![\pi, C]\!]_{\text{qviol}} = 0 \Leftrightarrow \pi$ is fully compliant ($\models_?$, $\models_{\top^t}$, or $\models_{\top^p}$).
- $[\![\pi, C]\!]_{\text{qviol}} > 0$ even when $\pi \models_{\top^t} C$ (through reparation) $\Rightarrow$ *cost of repair visible*.
- Monotonically increasing along prefixes.

Monitor: Mono-Quantitative Mealy machine $\text{QMC}(C)$.

$$\lambda(q, A) := [\![\langle A\rangle,\ q]\!]_{\text{qviol}}$$

Theorem: $\text{Score}(\text{QMC}(C), \pi) = [\![\pi, C]\!]_{\text{qviol}}$.

Quantitative Blame Semantics — Vector Domain $\mathbb{N}^2$

Refinement: Replace scalar $\mathbb{N}$ by agent-indexed vectors.

$$[\![\cdot]\!]_{\text{qblame}} : \Gamma^* \times \text{TACNL} \to \mathbb{N}^2$$

Instantaneous blame:

$$[\![\langle A\rangle,\ C]\!]_{\text{qblame}} := \begin{cases} [\![\langle A\rangle, C_1]\!]_{\text{qblame}} + [\![\langle A\rangle, C_2]\!]_{\text{qblame}} & C = C_1 \wedge C_2 \\ (1,0) & \langle A\rangle \vDash_{\bot_1^t} \text{LH}(C) \\ (0,1) & \langle A\rangle \vDash_{\bot_2^t} \text{LH}(C) \\ (0,0) & \text{otherwise} \end{cases}$$

No $(1,1)$ at literal level: blame verdicts are mutually exclusive per literal. Joint blame $(1,1)$ arises only through *conjunction* over distinct literals.

Example: $\mathbf{P}_1(\text{OCC}) \wedge \mathbf{O}_1(\text{PAY\_R})$, $A = \{\text{OCC}^{(1)}\}$:

$$[\![\langle A\rangle,\ \mathbf{P}_1(\text{OCC})]\!]_{\text{qblame}} = (0,1)$$
$$[\![\langle A\rangle,\ \mathbf{O}_1(\text{PAY\_R})]\!]_{\text{qblame}} = (1,0)$$
$$[\![\langle A\rangle,\ C_2 \wedge C_3]\!]_{\text{qblame}} = (1,1)$$

Decomposition lemma:

$$[\![\pi, C]\!]_{\text{qviol}} = n_1 + n_2 \quad \text{where} \quad [\![\pi, C]\!]_{\text{qblame}} = (n_1, n_2)$$

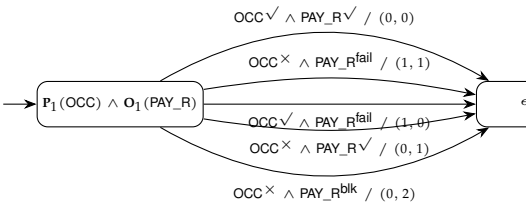
$\Rightarrow$ Blame is a *finer view* of the same violation count.

Double-Quantitative Monitor — QBM(C)

Definition: Mealy machine with output in $\mathbb{N}^2$.

$$\mathsf{QBM}(C) := (Q,\ q_0,\ \Gamma,\ \delta,\ \lambda_{\mathbb{N}^2})$$

- $Q := \mathsf{RRC}(C), \quad q_0 := C$
- $\delta(q, A) := \mathsf{Prog}(\langle A \rangle, q)$
- $\lambda_{\mathbb{N}^2}(q, A) := [\![\langle A \rangle, q]\!]_{\mathsf{qblame}}$



Fragment of $\mathsf{QBM}(\mathbf{P}_1(\mathsf{OCC}) \wedge \mathbf{O}_1(\mathsf{PAY\_R}))$.
Note: $(0, 2)$ when *both* violations blamed on agent 2.

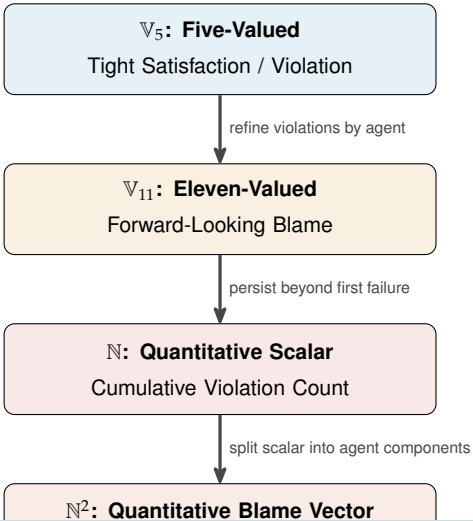
Blame execution score:

$$\mathsf{Score}(\mathsf{QBM}(C), \pi) = \sum_{i=0}^{|\pi|-1} \lambda_{\mathbb{N}^2}(q_i, A_i)$$

Conformance theorem:

$$\mathsf{Score}(\mathsf{QBM}(C), \pi) = [\![\pi, C]\!]_{\mathsf{qblame}}$$

The Full Semantic Hierarchy



Moore $\mathcal{M}_5$

Moore $\mathcal{M}_{11}$

Mealy QMC(C)

Summary of Contributions

Conflict Analysis (μMTNL)

1. Formal machinery: interval-set logic, DPNF, conflict calculus.
2. Exact characterisation: unsat $\Leftrightarrow \mathsf{C}_\delta(\phi) = 1$.
3. Conflict density metric $\mathsf{C}_\delta(\phi) \in [0, 1]$.
4. Sound & complete conflict-free rewriting $\mathsf{CFR}(\phi)$.
5. Complexity bounds & conciseness analysis.

Contract Monitoring (TACNL)

1. Attempt-based interaction model.
2. TACNL logic with repetition, guards, triggers, reparation.
3. 5-valued tight satisfaction semantics + Moore monitors.
4. 11-valued forward blame semantics + blame monitors.
5. Quantitative scalar ($\mathbb{N}$) and vector ($\mathbb{N}^2$) blame monitors.
6. Full pipeline: denotational $\rightarrow$ operational, with conformance proofs.

Outlook & Future Work

Extending μMTNL**:**

- Actions with *duration*.
- Conditional & permission literals.
- Defeasibility mechanisms.
- Interaction frames (prevention / enabling / interference between actions).

Extending TACNL**:**

- Constructive proof of RRC(C) finiteness.
- n-agent generalisation ($\mathbb{N}^n$ blame vectors).
- Integration of static conflict analysis and dynamic monitoring.
- Tool development: solver + runtime monitor.

Transferability: The methodology — normalise, detect, quantify, synthesise — is applicable to other timed deontic formalisms and contract languages.

Backup Slides

- Luco and LAP definition with intuitive explanation (pictures and formulas)
- examples of repair (positive and negative)
- conflict calculus rules
- from Metric-Timed to Collaborative Periodic Synchronized Set Traces (the example in steps with arrow explanation)
-